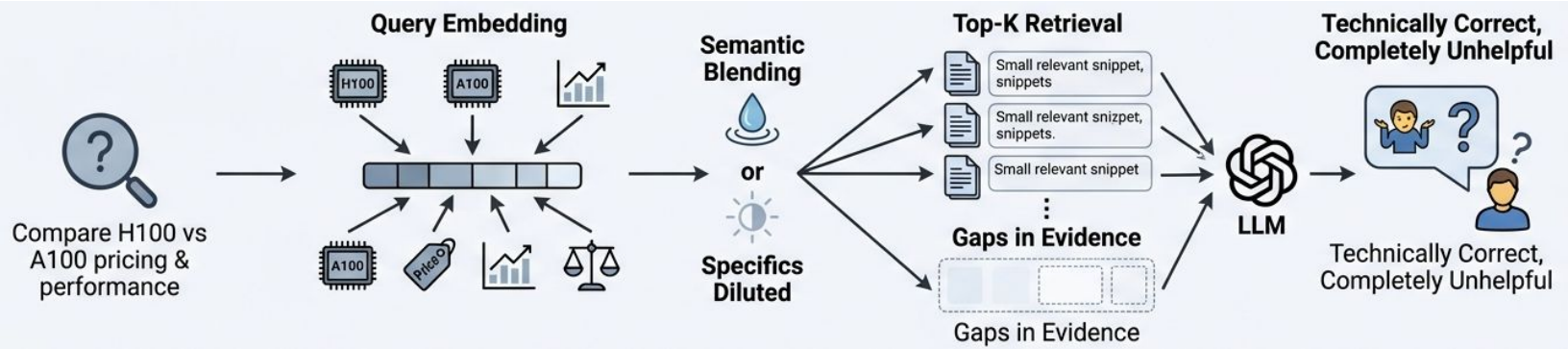
U.S. General Services Administration

Chapter 6.6: Query Decomposition and Adaptive Retrieval

US General Service Administration

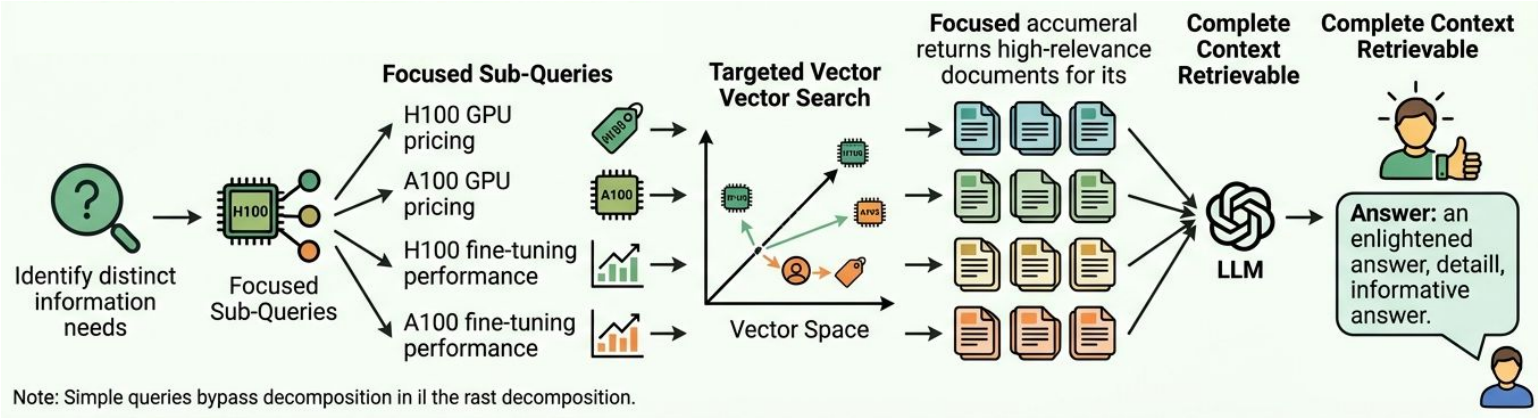
AI Community of Practice - March 2026

The Single-Vector Problem



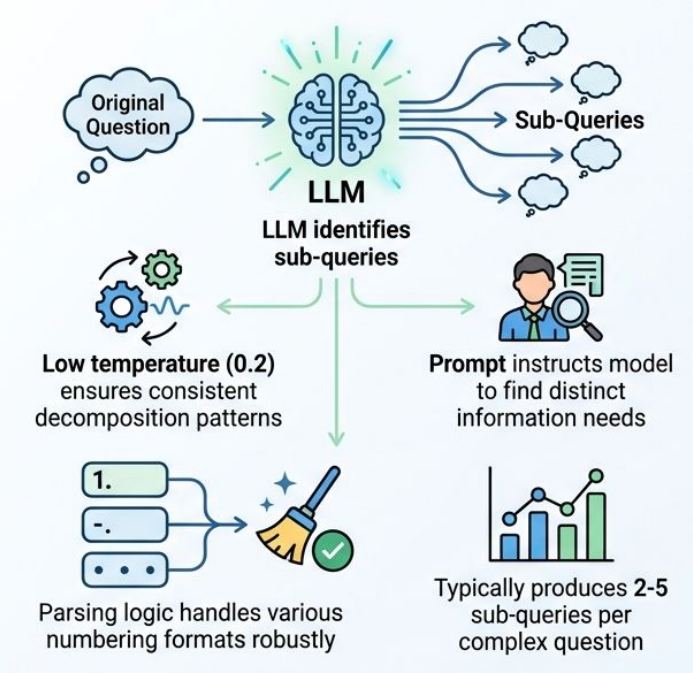
- Standard RAG embeds entire query into one vector
- Embedding captures a semantic blend that dilutes specificity
- Top-k retrieval returns partially relevant, incomplete context
- LLM generates answers from gaps, not from evidence
- "Technically correct, completely unhelpful" is the failure mode

Decomposition Strategy



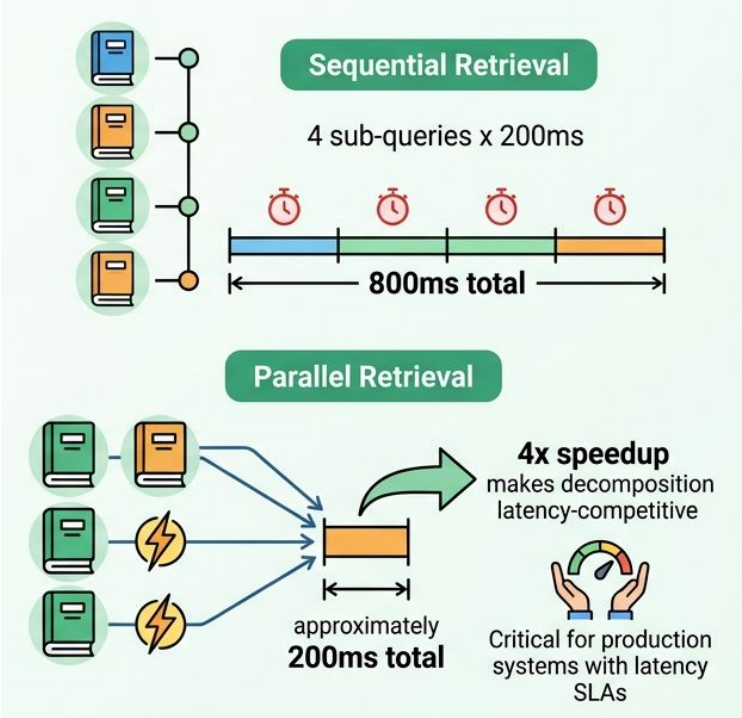
- Identify distinct information needs within the original query
- Generate focused sub-queries for each component
- Each sub-query targets a specific region of vector space
- GPU comparison decomposes into four precise sub-queries
- Simple queries bypass decomposition entirely

LLM-Powered Decomposition Implementation



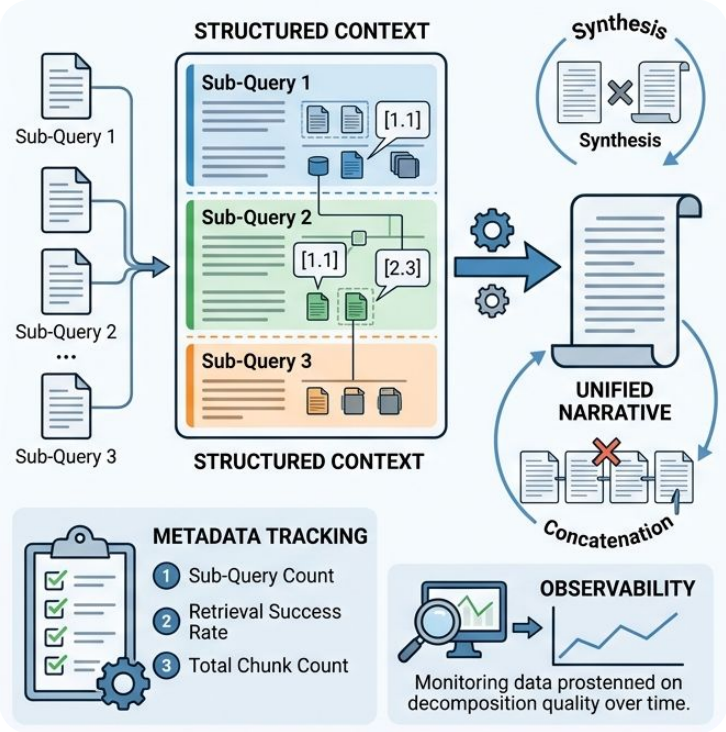
- The LLM itself identifies sub-queries from the original question
- Low temperature (0.2) ensures consistent decomposition patterns
- Prompt instructs model to find distinct information needs
- Parsing logic handles various numbering formats robustly
- Typically produces 2-5 sub-queries per complex question

Parallel Retrieval for Performance



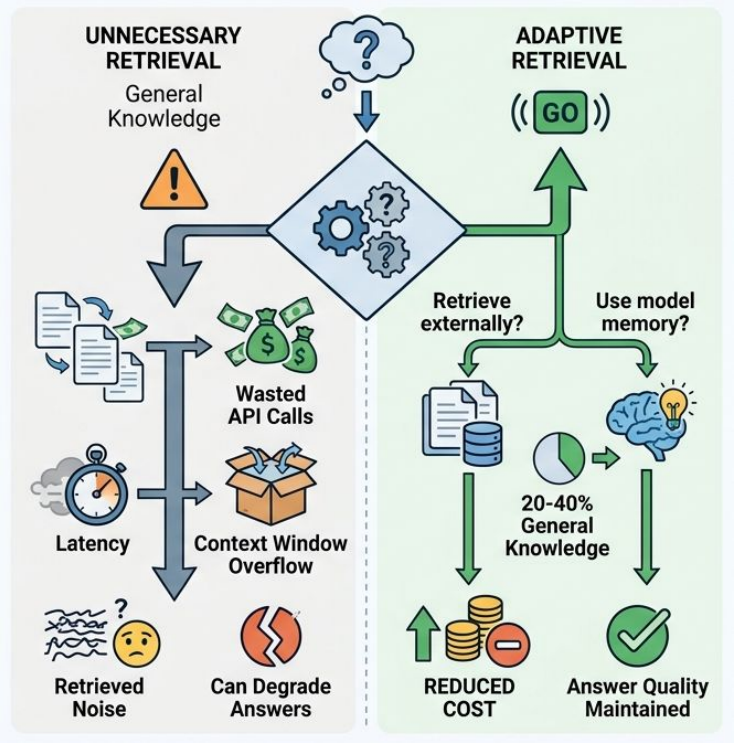
- Sequential retrieval: 4 sub-queries x 200ms = 800ms total
- Parallel retrieval with asyncio.gather(): approximately 200ms total
- Latency equals the slowest single retrieval, not the sum
- 4x speedup makes decomposition latency-competitive
- Critical for production systems with latency SLAs

Answer Synthesis from Multiple Contexts



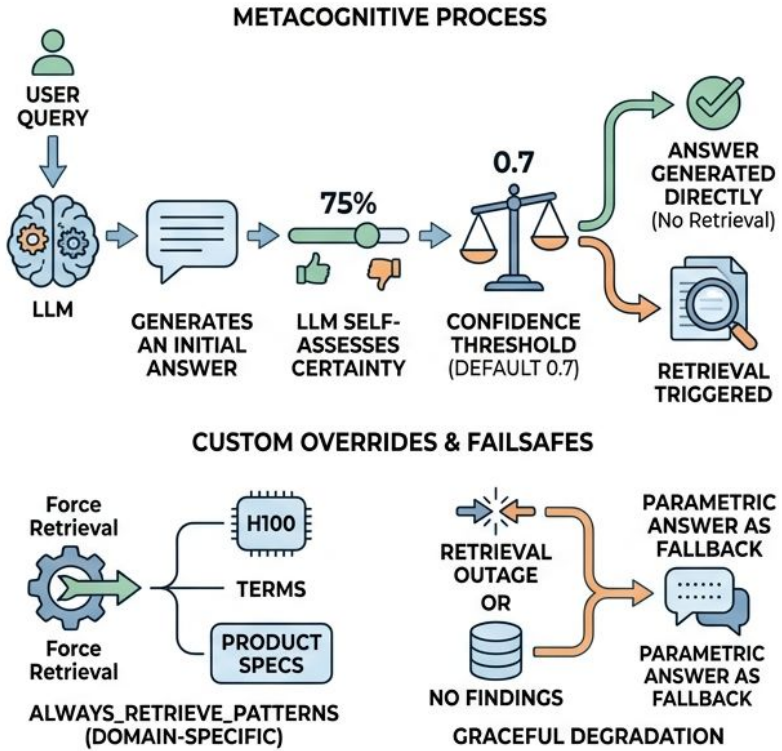
- Structured context with clear section boundaries per sub-query
- Hierarchical references like [1.1] enable precise citation
- Synthesis prompt emphasizes coherent narrative, not concatenation
- Metadata tracks sub-query count, retrieval success, and chunk totals
- Observability enables monitoring decomposition quality over time

Adaptive Retrieval -- Not Every Query Needs It



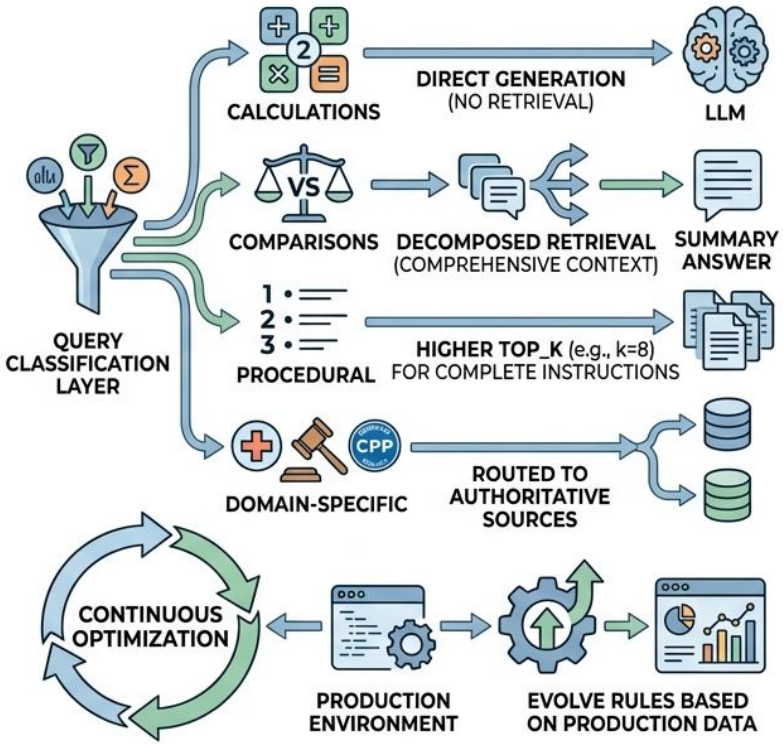
- 20-40% of production queries target general knowledge
- Unnecessary retrieval wastes API calls, latency, and context window
- Retrieval noise can actually degrade correct parametric answers
- Adaptive retrieval decides: retrieve externally or use model memory
- Reduces cost while maintaining answer quality

Confidence-Based Retrieval Decisions



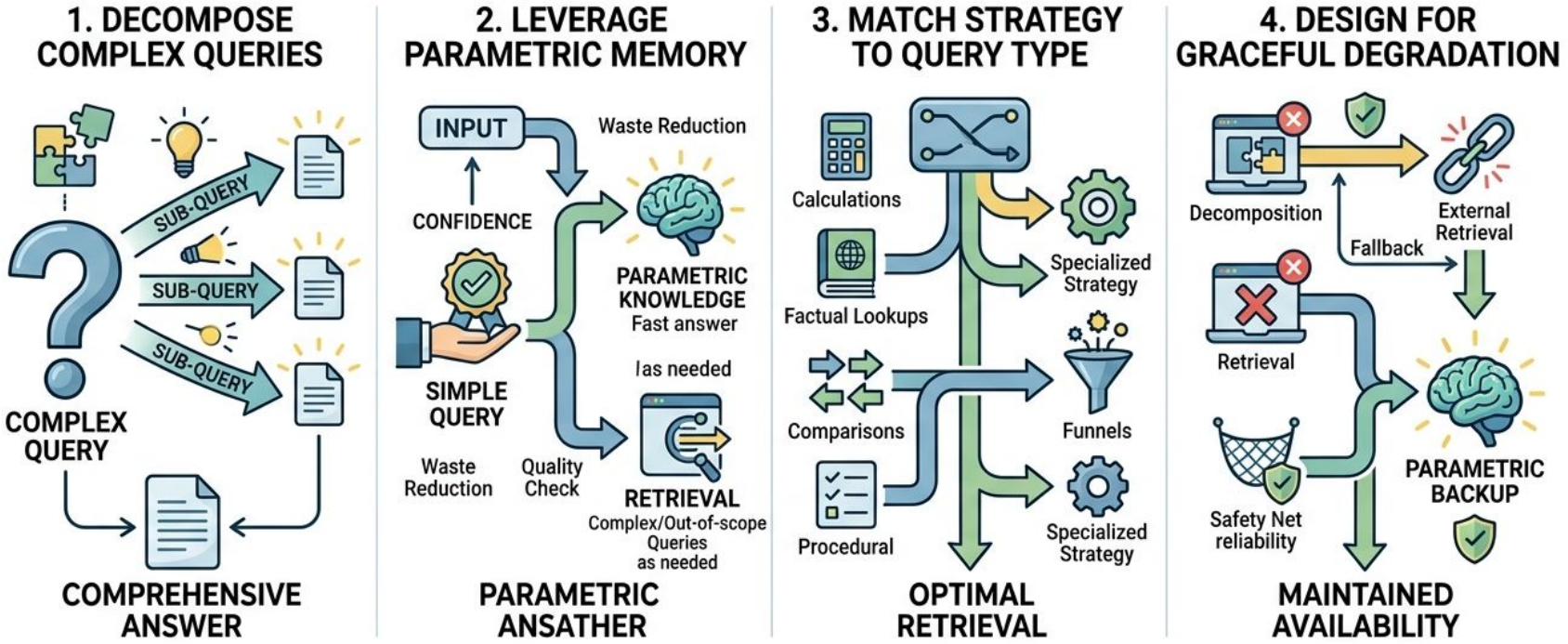
- LLM generates initial answer, then rates its own certainty
- Threshold (default 0.7) balances false positives vs false negatives
- always_retrieve_patterns force retrieval for domain-specific queries
- Fallback returns parametric answer when retrieval finds nothing
- Graceful degradation maintains service during retrieval outages

Pattern-Based Query Routing

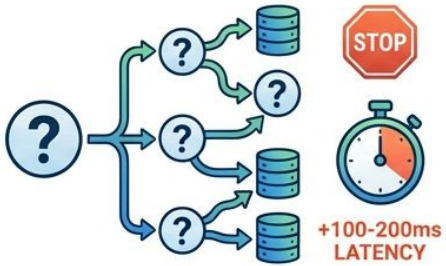


- Query classification routes different types to optimal strategies
- Calculations go to direct generation -- no retrieval needed
- Comparisons trigger decomposed retrieval automatically
- Procedural queries use higher top_k for complete instructions
- Routing rules must evolve based on production performance data

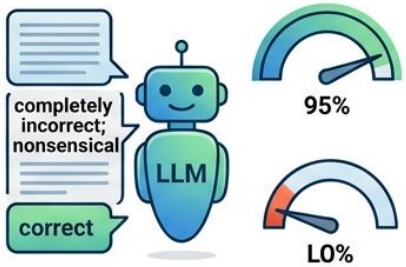
Four Principles of Advanced Retrieval



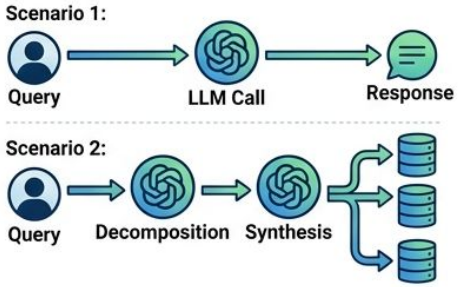
Common Pitfalls to Avoid



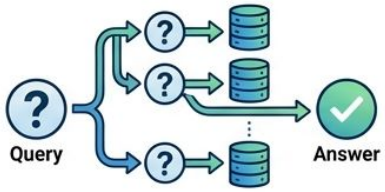
AVOID OVER-DECOMPOSITION |
Adds significant latency without quality gain



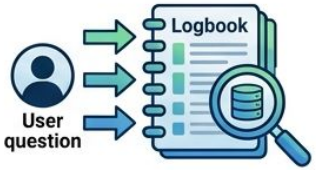
BEWARE OF CONFIDENCE SCORES |
Not perfectly calibrated; may over-trust wrong answers



CONSIDER COST & LLM CALLS |
Decomposition doubles LLM generations; measure if accuracy justifies overhead



STALE ROUTING RULES
Rules decay as query patterns shift over time



ALWAYS LOG ROUTING DECISIONS
Vital for identifying patterns & post-hoc analysis

Key Takeaways

- Query decomposition transforms one mediocre retrieval into multiple excellent ones
- Adaptive retrieval saves 20-40% of unnecessary retrieval operations
- Parallel execution and smart routing keep latency under control
- These techniques complete Part 6's knowledge integration toolkit
- Next: Chapter 7.1A -- implementing guardrails with NeMo on the NVIDIA platform